BusinessX

# Database Guide

Tables, relationships, indexes and why each table exists · MySQL · schema as of 13 October 2026 (branch `development`)

| 102 | ~30 | 0 | 4 |
|---|---|---|---|
| tables created by migrations | tables used by the current app | database foreign keys (relations enforced in code) | member profile types |

## 1. How the schema is organised

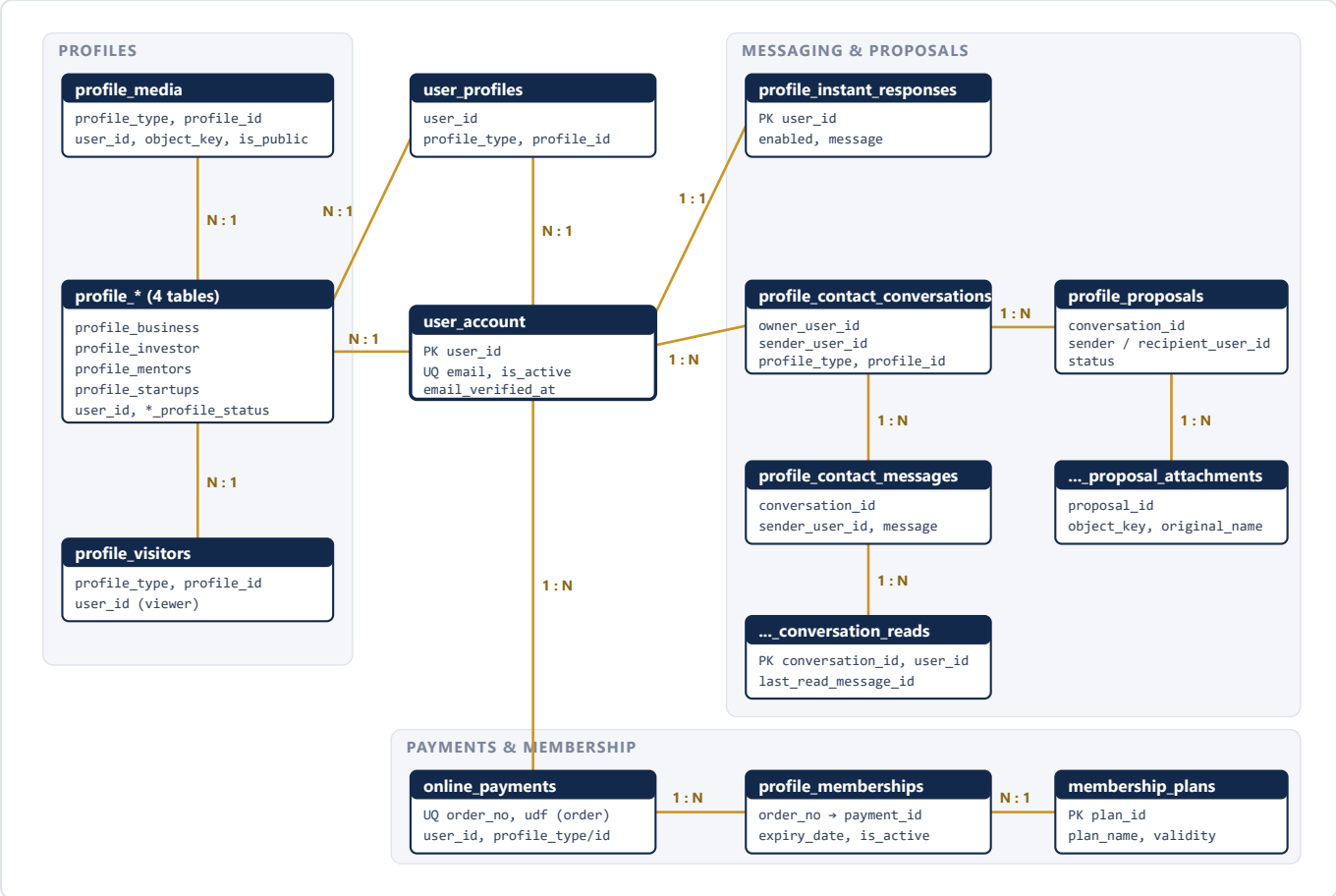
- **Legacy origin.** Most tables come from the previous PHP system and keep their names, keys and tinyint flags. New features (messaging, proposals, media, chatbot, notifications) use newer tables with Laravel conventions (`id`, `created_at`, `updated_at`).
- **One account, many profiles.** A person has one `user_account`. Each of the four profile tables links back to it through `user_id`, so one account can own several profiles of different types.
- **Polymorphic profile references.** Tables that can point at any profile type store a pair: **profile type + profile id**. Two encodings exist:

| Profile type | Numeric code | String | Table · primary key | Status column (1 = active) |
|---|---|---|---|---|
| Business | 1 | `business` | `profile_business.business_id` | `business_profile_status` |
| Investor | 2 | `investor` | `profile_investor.investor_id` | `inv_profile_status` |
| Mentor | 4 | `mentor` | `profile_mentors.mentor_id` | `mentor_profile_status` (+ `deleted_at`) |
| Startup | 7 | `startup` | `profile_startups.startup_id` | `startup_profile_status` |

Numeric codes are used by `user_profiles`, `profile_visitors`, `online_payments` and `profile_memberships`. String names are used by `profile_contact_conversations`, `profile_proposals` and `profile_media`.

> **No foreign keys**
> The legacy schema has no database-level foreign keys. Every relationship below is *logical*: the application joins on it and checks it in queries (for example, ownership is always filtered by `user_id`). Deleting rows by hand can leave orphans, so delete children first.

## 2. Relationship diagram (active tables)




Gold lines are logical relationships (no database constraint). Not drawn, to keep it readable: reference data (`industry_categories`, `ind_pref_business`, `bx_cities`), content (`bx_articles` → `bx_author`, `articles_comments`) and framework tables. They are all listed in section 4.

## 3. Active tables: purpose and indexes

| Table | Why it exists | Keys and indexes |
|---|---|---|
| **Accounts** | | |
| **user_account** | One row per person; the login model (`UserAccount`). Holds credentials, contact details, verification and active flags. | `PK user_id`<br>`UQ email`<br>`UQ user_rand_id` |
| **user_profiles** | Legacy index of which profiles an account owns (type code + profile id + profile key). Written at registration for compatibility with the old system. | `PK user_prof_id` |
| **password_reset_tokens** | Hashed, single-use password reset tokens (Laravel broker). | `PK email` |
| **sessions** | Database-backed sessions; lets the app sign a user out everywhere. | `PK id`<br>`IX user_id`<br>`IX last_activity` |
| **Profiles** | | |
| **profile_business** | Business listings (for sale, seeking investment or loans): company, financials, location, images, membership flags. | `PK business_id`<br>`IX user_id`<br>`IX (status, created_at, business_id)` |
| **profile_investor** | Investor profiles: investment range, preferences, company details. | `PK investor_id`<br>`IX user_id`<br>`IX (status, created_at, investor_id)` |
| **profile_mentors** | Mentor profiles: expertise, experience, location (soft-deleted via `deleted_at`). | `PK mentor_id`<br>`IX user_id`<br>`IX (status, created_at, mentor_id)` |
| **profile_startups** | Startup profiles: funding ask, stage, industry (`industry_sector` = category id). | `PK startup_id`<br>`UQ startup_profile_str`<br>`IX user_id`<br>`IX (status, created_at, startup_id)`<br>`IX (status, industry_sector)` |
| **profile_business_mgmt** | Key contact / management person attached to a business profile. | `PK business_mgmt_id` |
| **profile_media** | Every uploaded image/document with its S3 key, visibility and owner. Lets private documents be served only to their owner via signed URLs. | `PK id`<br>`IX (user_id, profile_type, profile_id)`<br>`IX (profile_type, profile_id)` |
| **profile_visitors** | One row per profile view by a signed-in member; powers the "most searched" view counts on the dashboard. | `PK visitor_id`<br>`IX (profile_type, profile_id)` |
| **Reference data** | | |
| **industry_categories** | Two-level industry tree (parent → sub-category) used for filters, footer links and home counters. | `PK cat_id`<br>`IX (parent_id, category_status)` |

| Table | Why it exists | Keys and indexes |
|---|---|---|
| **ind_pref_business** | Maps business profiles to industry categories (many-to-many), used by industry filters and home counts. | PK business_ind_pref_id<br>IX (business_profile_id, profile_status, parent_category_id)<br>IX (business_profile_id, profile_status, sub_category_id) |
| **bx_cities** | Allowed UK cities (`country = 5`); validates city fields and drives autocomplete and the home city section. | PK id |
| **country** | Country list for location fields. | PK country_id |
| **Messaging & proposals** | | |
| **profile_contact_conversations** | A thread between a profile's owner and an interested member about one profile. | PK id<br>IX (owner_user_id, updated_at)<br>IX (sender_user_id, updated_at)<br>IX (profile_type, profile_id) |
| **profile_contact_messages** | Messages inside a conversation (also broadcast live). | PK id<br>IX (conversation_id, created_at)<br>IX (conversation_id, id) |
| **profile_contact_conversation_reads** | Per-user "last read message" marker; drives unread counts. | PK (conversation_id, user_id)<br>IX (user_id, last_read_message_id) |
| **profile_instant_responses** | Owner's optional automatic first reply to new contacts (one per user). | PK user_id |
| **profile_proposals** | Formal offers sent by a profile owner to an interested member; pending / accepted / declined. | PK id<br>IX (conversation_id, created_at)<br>IX (sender_user_id, status, created_at)<br>IX (recipient_user_id, status, created_at) |
| **profile_proposal_attachments** | Private files (S3) attached to a proposal. | PK id<br>IX (proposal_id, created_at) |
| **notifications** | Laravel database notifications (header bell, activity page). | PK id (uuid)<br>IX (notifiable_type, notifiable_id) |
| **Payments & membership** | | |
| **online_payments** | Every checkout attempt: payer, plan, server-side amount, Razorpay order id (`udf`) and payment id, status 0/1/2. | PK payment_id<br>UQ order_no<br>UQ razorpay_payment_id<br>IX udf |
| **profile_memberships** | History of paid memberships per profile with activation/expiry; the My Plan page reads it. | PK membership_id<br>IX (user_id, profile_type, profile_id) |
| **membership_plans** | Plan catalogue (names, validity, credits) used to label memberships. | PK plan_id |

| Table | Why it exists | Keys and indexes |
|---|---|---|
| **Content & marketing** | | |
| **bx_articles** | Articles / insights (maintained by the admin team). | PK article_id<br>IX (status, created_at, id)<br>IX (status, views, id)<br>IX (status, comments, id) |
| **bx_author** | Article authors. | PK author_id |
| **articles_comments** | Member comments on articles; shown after moderation (`comment_status = 1`). | PK comment_id<br>IX (article_id, comment_status, created_at) |
| **testimonials** | Client quotes for the home page, ordered and switchable. | PK id<br>IX (is_active, sort_order) |
| **businessex_newsletter** | Newsletter subscribers with double opt-in status (P / S / U). | PK newsletter_id<br>UQ email |
| **businessex_contactus** | Contact-us form submissions. | PK contact_id |
| **chatbot_knowledge_base** | Keyword → answer pairs for the site chatbot. | PK id<br>UQ intent |
| **chatbot_leads** | Contact details captured by the chatbot. | PK id<br>IX created_at |
| **Framework (Laravel)** | | |
| **cache, cache_locks** | Database cache store (home page, schema metadata, chatbot answers). | PK key<br>IX expiration |
| **jobs, job_batches, failed_jobs** | Queue for emails and notifications; failed jobs for inspection. | PK id<br>IX queue<br>UQ failed_jobs.uuid |
| **migrations** | Which migrations have run. | PK id |

## 4. Relationships

| Child column | Refers to | Type | Notes |
|---|---|---|---|
| profile_business / _investor / _mentors / _startups.user_id | user_account.user_id | N : 1 | Owner of the profile. |
| user_profiles.user_id, (profile_type, profile_id) | user_account, profile_* | N : 1 | Numeric type code. |
| profile_business_mgmt.business_profile_id | profile_business.business_id | N : 1 | |
| profile_media (profile_type, profile_id), user_id | profile_*, user_account | N : 1 | String type; owner check on download. |
| profile_visitors (profile_type, profile_id) | profile_* | N : 1 | Numeric type; `user_id` is the viewer. |
| ind_pref_business.business_profile_id | profile_business.business_id | N : 1 | With parent/sub category ids → `industry_categories.cat_id`. |
| industry_categories.parent_id | industry_categories.cat_id | N : 1 | Self-reference; 0 = top level. |

| Child column | Refers to | Type | Notes |
|---|---|---|---|
| profile_startups.industry_sector | industry_categories.cat_id | N : 1 | Business uses free-text sector instead. |
| bx_cities.country | country.country_id | N : 1 | 5 = United Kingdom. |
| profile_contact_conversations.owner_user_id / sender_user_id | user_account.user_id | N : 1 | Plus (profile_type, profile_id) of the profile discussed. |
| profile_contact_messages.conversation_id | profile_contact_conversations.id | N : 1 | sender_user_id → user_account. |
| profile_contact_conversation_reads (conversation_id, user_id) | conversations, user_account | N : 1 | last_read_message_id → messages.id. |
| profile_instant_responses.user_id | user_account.user_id | 1 : 1 | Primary key is the user. |
| profile_proposals.conversation_id | profile_contact_conversations.id | N : 1 | sender/recipient → user_account. |
| profile_proposal_attachments.proposal_id | profile_proposals.id | N : 1 | |
| notifications (notifiable_type, notifiable_id) | user_account.user_id | N : 1 | Laravel polymorphic. |
| online_payments.user_id, (profile_type, profile_id) | user_account, profile_* | N : 1 | 0 until a guest payment is claimed at registration. |
| profile_memberships.order_no | online_payments.payment_id | N : 1 | Despite its name it stores the payment id. |
| profile_memberships.membership_type | membership_plans.plan_id | N : 1 | |
| bx_articles.author_id | bx_author.author_id | N : 1 | |
| articles_comments.article_id | bx_articles.article_id | N : 1 | |
| sessions.user_id, password_reset_tokens.email | user_account | N : 1 | Framework tables. |

## 5. Why these indexes

| Index | Query it serves |
|---|---|
| profile_* (status, created_at, id) | Directory listings and home sliders: "active profiles, newest first", paginated. |
| profile_* (user_id) | Dashboard: "my profiles", and every ownership check. |
| profile_startups (status, industry_sector) | Startup listing filtered by industry. |
| ind_pref_business (business_profile_id, profile_status, parent/sub category) | Business industry filter and home industry counters. |
| industry_categories (parent_id, category_status) | Building the category tree for filters and the footer. |
| conversations (owner_user_id, updated_at) and (sender_user_id, updated_at) | Inbox: "my conversations, most recent first" for either side. |
| messages (conversation_id, id) / (conversation_id, created_at) | Loading a thread, newest messages, latest message per conversation, unread counts. |

| Index | Query it serves |
|---|---|
| `conversation_reads PK (conversation_id, user_id)` | One marker per user per thread; makes the read update an atomic upsert. |
| `proposals (sender\|recipient_user_id, status, created_at)` | "Proposals sent / received" lists with the status filter, newest first. |
| `profile_media (user_id, profile_type, profile_id)` | Media for one owned profile; owner checks on download. |
| `profile_visitors (profile_type, profile_id)` | Dashboard view counts. Verified with EXPLAIN: full scan of 5,000 rows → 2 rows. |
| `articles_comments (article_id, comment_status, created_at)` | Approved comments for one article, newest first. EXPLAIN: 5,000-row scan + filesort → ~25 rows. |
| `bx_articles (status, created_at\|views\|comments, id)` | Article list sorted by newest, most viewed or most commented. |
| `online_payments UQ order_no, UQ razorpay_payment_id, IX udf` | Look-ups by order number and Razorpay order id; the unique keys also stop one payment being applied twice. |
| `profile_memberships (user_id, profile_type, profile_id)` | Deactivating the previous membership and showing My Plan. |
| `UQ user_account.email, businessex_newsletter.email` | Prevent duplicate accounts and subscriptions; fast login look-up. |

> **Not indexed on purpose**
> Free-text search uses `LIKE '%term%'` across several columns, which a normal index cannot speed up. If search becomes slow as data grows, add FULLTEXT indexes or a search engine instead of more B-tree indexes.

## 6. Legacy tables (kept, not used by the current app)

About 70 tables were migrated from the old system for data preservation and possible future features. The current code does not query them:

| Group | Tables |
|---|---|
| Other profile types | `profile_broker, profile_incubators, profile_lenders and their *_prof_exp, *_mgmt, *_fund_raising tables` |
| Old preferences | `ind_pref_* (except ind_pref_business), loc_pref_*, mentor_categories, mentor_expertise` |
| Old contact system | `contact_* (8 tables), request_contact, conversation_reply, conversation_reply_attachment` |
| Old content | `bx_news, news, news_list, news_comments, article_news_images, bex_shorts, bx_industryreports, content_tags, content_tags_assigned, seo, bx_dfp_banner` |
| Other legacy | `admin_user, admin_profile_rejected, bookmarks, business_bookmarks, business_images, startup_images, bx_services, bx_coupon, fi_investors, hongkong, mobile_verification, password_resets, businessex_riseconference, users` |

Before dropping any of them, confirm with the admin team that no external tool (for example an admin panel) still reads them, and take a backup.

## 7. Recommendations

- **Add foreign keys gradually**, starting with payments, memberships, messaging and proposals, after removing orphaned rows.
- **Use one profile-type encoding.** New tables use strings (`business`), older ones numeric codes; pick one for future tables and map the other.
- **Store money as `DECIMAL`**: `online_payments.amount`, `profile_memberships.amount` and `membership_plans.plan_amount` are strings today.
- **Keep `profile_visitors` small**: count one view per viewer per profile per day, or aggregate old rows.
- **Archive or drop legacy tables** once confirmed unused, to reduce backup size and confusion.
- **Every schema change goes through a migration** that runs on MySQL and SQLite and has a working rollback; run `php artisan migrate` (it also refreshes the app's schema cache).